

# Arbitrary Precision Arithmetic Library

Pathri Vidya Praveen - CS24BTECH11047

May 2, 2025

## Contents

|          |                                 |          |
|----------|---------------------------------|----------|
| <b>1</b> | <b>Design Overview</b>          | <b>2</b> |
| 1.1      | UML Class Diagram . . . . .     | 2        |
| <b>2</b> | <b>AInteger Class</b>           | <b>2</b> |
| <b>3</b> | <b>AFloat class</b>             | <b>4</b> |
| <b>4</b> | <b>Limitations</b>              | <b>5</b> |
| <b>5</b> | <b>Verification Approach</b>    | <b>5</b> |
| <b>6</b> | <b>Key Learnings</b>            | <b>5</b> |
| <b>7</b> | <b>Library Usage and README</b> | <b>6</b> |

# 1 Design Overview

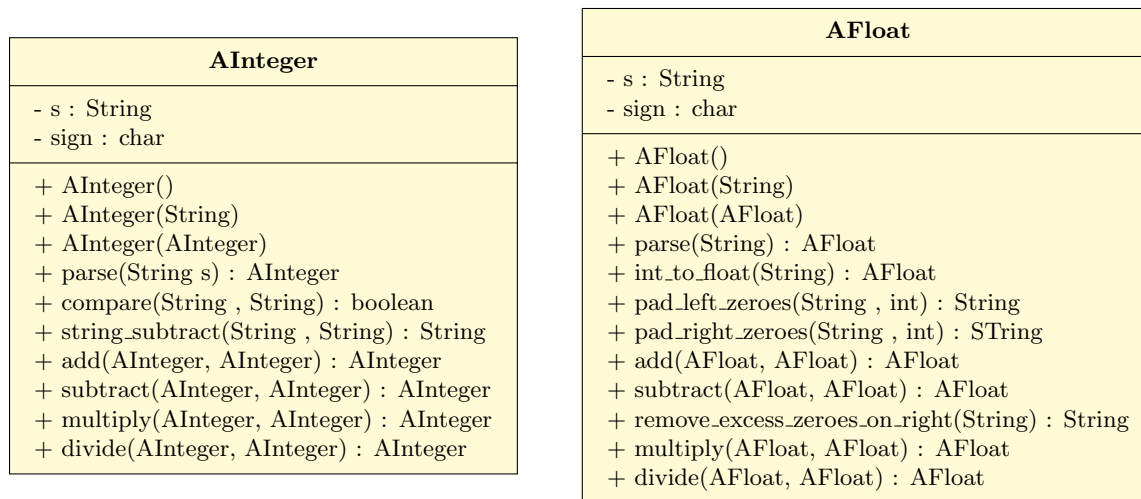
This project focusses on implementation of an infinite/arbitrary-precision arithmetic library in Java using OOP concepts. Given two strings as input, you need to add support for addition, subtraction, multiplication and division, for both integer and float data types. The answer should be printed on the terminal screen.

Using these classes, one should be able to compute any expression involving arbitrarily large integers and arbitrarily large floating point precision numbers with arbitrary precision.

All the operations that you implement should preserve the complete information of the number and should not introduce any kind of round-off errors.

The actual decimal precision may vary depending on how your MyInfArith handles decimal scale/rounding, we will follow Java's BigDecimal class and truncate at 30 decimal digits.

## 1.1 UML Class Diagram



## 2 AInteger Class

### • Public Fields:

- `public String s`  
Stores the digits of the arbitrary-length integer in string form.
- `public char sign`  
Stores the sign of the integer. Either '+' or '-'. Zero is considered positive.

### • Constructors:

- `AInteger()`  
Default constructor. Initializes the number to 0 with a positive sign.
- `AInteger(String s)`  
Initializes an instance from a string. Validates that the string is non-null, non-empty, and contains only digits (optionally preceded by '+' or '-').
- `AInteger(AInteger other_copy)`  
Copy constructor. Creates a deep copy of another AInteger instance.

### • Static Method:

- `static AInteger parse(String s)`  
Parses a string and returns a new AInteger instance from it.

- `static AInteger add(AInteger s1, AInteger s2):`
  - Handles all sign cases:
    - \* `(+) + (+)`: Direct digit-wise addition.
    - \* `(+) + (-)`: Converts to subtraction.
    - \* `(-) + (+)`: Converts to subtraction.
    - \* `(-) + (-)`: Adds magnitudes and keeps negative sign.
  - Performs manual digit-by-digit addition from right to left using carry.
  - Removes leading zeroes from result.
- `static boolean compare(String a, String b):`
  - Compares two positive numeric strings.
  - Returns `true` if `a > b`, else `false`.
- `static String string_subtract(String a, String b):`
  - Assumes `a >= b`, and both are positive.
  - Performs digit-wise subtraction from right to left using borrow.
- `static AInteger subtract(AInteger s1, AInteger s2):`
  - Handles all sign combinations:
    - \* `(+) - (+)`: Uses magnitude comparison.
    - \* `(-) - (+)`: Converts to addition, keeps sign.
    - \* `(+) - (-)`: Converts to addition.
    - \* `(-) - (-)`: Converts to `b - a`.
  - Uses `compare()` to determine magnitude.
  - Uses `string_subtract()` accordingly.
- `static AInteger multiply(AInteger s1, AInteger s2):`
  - Handles all sign combinations.
  - Uses long multiplication (school method).
  - For each digit in the second number (from right), multiplies with full first number by repeated addition.
  - Appends necessary zeros (like multiplying by powers of 10).
  - Accumulates all partial products using `add()`.
- `class DivisionByZeroException extends ArithmeticException:`
  - Custom exception thrown when division by zero is attempted.
  - Overrides constructor to give a clear message.
- `static AInteger divide(AInteger s1, AInteger s2):`
  - Handles all sign combinations.
  - Uses long division (school method).
  - While division by dividend with divisor, it takes digit by digit while division and then divides the remainder with divisor and add the quotient to the result quotient and then add the next digit to the remainder and continue the process until you are left with remainder less than divisor

### 3 AFloat class

- **Public Fields:**

- `public String s`  
Stores the digits of the arbitrary-length floating point in string form.
- `public char sign`  
Stores the sign of the integer. Either '+' or '-'. Zero is considered positive.

- **Constructors:**

- `AFloat()`  
Default constructor. Initializes the number to 0.0 with a positive sign.
- `AFloat(String s)`  
Initializes an instance from a string. Validates that the string is non-null, non-empty, and contains only digits and at most one decimal point (optionally preceded by '+' or '-').
- `AFloat(AFloat other_copy)`  
Copy constructor. Creates a deep copy of another AFloat instance.

- **Static Method:**

- `static AFloat parse(String s)`  
Parses a string and returns a new AFloat instance from it.

- `static AFloat add(AFloat s1, AFloat s2):`

- First remove the decimal point from both the strings.
- Then perform addition using AInteger addition
- Then add the decimal point in the appropriate location

- `static AFloat subtract(AFloat s1, AFloat s2):`

- Remove the decimal point from the strings
- Then do the normal AInteger subtraction
- Then finally place the decimal point back into the final result string in the appropriate position.

- `static AFloat multiply(AFloat s1, AFloat s2):`

- Remove the decimal point from both the strings and then do the normal AInteger multiplication and finally place the decimal point back into the final result string.

- `class DivisionByZeroException extends ArithmeticException:`

- Custom exception thrown when division by zero is attempted.
- Overrides constructor to give a clear message.

- `static AFloat divide(AFloat s1, AFloat s2):`

- Handles the exception when the denominator is zero and cases when numerator is zero or numerator is equal to denominator.
- Remove the decimal points from both the numerator and denominator and then multiply the numerator with  $10^{30}$  and then do AInteger division and then finally place the decimal point back in the final result string and then output it.

- `AFloat int_to_float(String s):`

- Converts an integer to floating point number by adding a .0 at the end of the number and return AFloat object.
- `String pad_left_zeroes(String s,int n):`
  - pads n number of zeroes to the left of string S
- `String pad_right_zeroes(String s,int n):`
  - pads n number of zeroes to the right of string s
- `String remove_excess_zeroes_on_right(String s):`
  - Removes excess zeroes on the right of a String

## 4 Limitations

- The library handles only base-10 string input.
- Performance is not optimized (e.g., multiplication is  $O(n^2)$ ).
- No exception hierarchy for user-defined overflow/underflow.
- This is a completely brute force approach not following any optimized algorithms. So this can be a good improvement in future.
- In the code , 0 in AInteger and 0.0 in AFloat are considered as positive by default because I am handling both magnitude and sign differently in 2 public variables. So this may lead to some errors if input is given like -0 or -0.0 .
- The Strings may not be taken of infinite length due to memory constraints of the machine. So it may fail if very very large numbers are given as input due to less memory to store such large strings. Also for numbers of order  $10^{10^8}$  , time taken to run the code may increase.

## 5 Verification Approach

All major operations were tested with:

- Test cases given in the problem statement.
- Randomized inputs
- Edge cases like zero, negative numbers, and large-length inputs
- For float division , initially I came across some bugs for  $1.2 / 200$ . Then I rectified the obtained index out of bound exception as my code needs to be updated the final result string to same length as that of number for keeping the decimal place back.
- Tested for numbers of different sign combinations and different lengths , numbers of very large difference in magnitude.

## 6 Key Learnings

- Low-level implementation of arithmetic operations.
- Importance of string manipulation and zero-padding in arbitrary-precision arithmetic.
- Debugging complex string-based number algorithms.
- Gained more conceptual clarity over encapsulation and other OOP concepts.
- Gained a good hands on experience with python script automation , git usage , docker image creation and usage of build systems like Ant.

## 7 Library Usage and README

- Refer to README.md file for more details.